

ELK Lab: Observability & Centralized Logging

Systems Reliability Engineering (SRE)

Agenda

1. Architecture Overview
2. The 6 Log Sources
3. ELK Stack Components
4. Setting Up the Stack
5. Windows Event Forwarding
6. Error Budgets & SLOs
7. Configuring Kibana
8. Learning Outcomes

The Challenge: Why Binary Monitoring Fails

Binary "up/down" monitoring is insufficient for modern systems

- **Cardinality explosion:** Millions of requests, each with unique latency fingerprints
- **Unknown unknowns:** You don't know where failures lurk until they happen
- **Statistical invisibility:** p99 latency spikes disappear in averages

Why p99 matters: If p99 = 3s, 1% of users wait 3 seconds. That's a lot of abandoned carts.

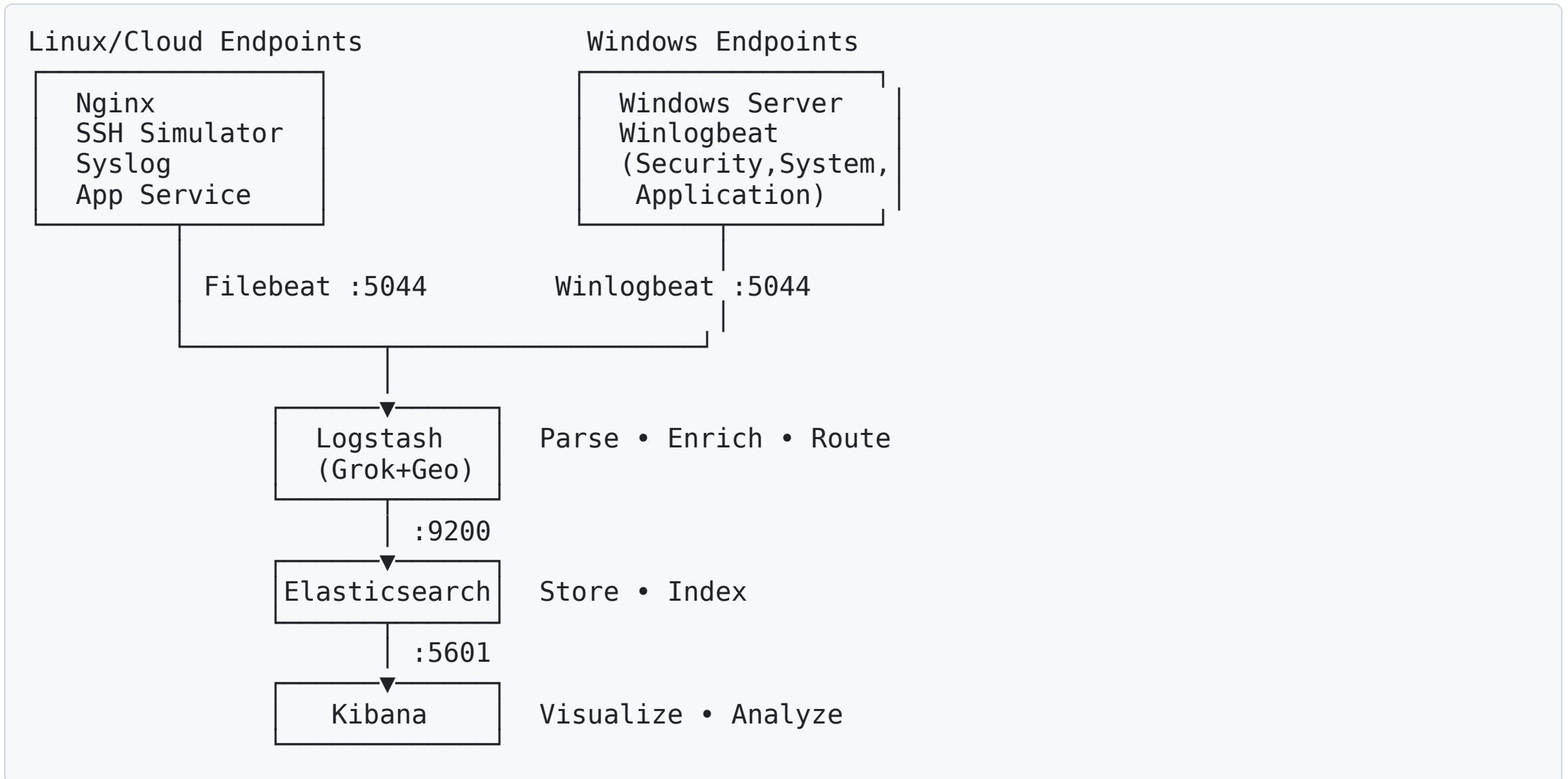
The Three Pillars of Observability

Pillar	Data Characteristic	E-Commerce Use Case
Logs	Discrete events, high cardinality	Debug failed checkout by order_id
Metrics	Aggregated numeric, low cardinality	Alert: error_rate > 2%
Traces	Per-request call graphs, very high cardinality	Find 3s delay source

How traces resolve bottlenecks:

- `trace_id` propagates across all services (HTTP headers)
- Each span = one operation with duration
- Waterfall view shows where time is spent

Architecture Overview



The 6 Log Sources

Source	Shipper	Format	Key Fields	Use Case
Nginx	Filebeat	Combined Apache Log	clientip, status, request	Web traffic
SSH	Filebeat	JSON	user, result, source_ip, geoip	Security
Syslog (Linux)	Filebeat	RFC5424	hostname, process, message	System events
Syslog (Cisco)	Filebeat	CEF	src_ip, dst_ip, severity	Network logs
App Service	Filebeat	JSON	method, endpoint, latency	App health

Log Source: Windows Event Logs

Shipper: Winlogbeat (official Elastic Beats for Windows)

Format: Windows Event XML (rendered as JSON by Winlogbeat)

Key Security Events:

Event ID	Channel	Description	Critical For
4624	Security	Account successfully logged on	Tracking valid logins
4625	Security	Account failed to log on	Detecting brute force
4634	Security	Logoff	Session tracking
4648	Security	Explicit credentials used	Silver ticket attacks

WEF Architecture: Push vs Pull

Feature	Source-Initiated (Push)	Collector-Initiated (Pull)
Scalability	HIGH - independent agents	LOW - collector bottleneck
Firewall	Complex - inbound rules	Simple - outbound only
Remote Laptops	IDEAL - tolerates disconnection	PROBLEMATIC - can't poll offline

Recommendation: Push for roaming devices, Pull for always-on servers

Windows Server 2025: IAKerb & Local KDC

IAKerb (Kerberos Armoring):

- Proxies Kerberos authentication between clients and KDC
- Eliminates NTLM fallback (weaker authentication)
- Required for "Protected Logon" in Windows Server 2025

Local KDC (Key Distribution Center):

- Embedded KDC runs directly on Windows Server 2025
- Enables offline authentication with cached credentials
- Hybrid/remote workers stay productive without DC connectivity

Why this matters:

- Remote workers on unstable VPNs get seamless auth

Reduces pressure on domain controllers

Forensic Prioritization

Exercise: Identify Windows Event IDs

Task	Event ID
Logon success	4624
Logon failure	4625

Why these matter:

- **4624** tracks legitimate user activity and normal working patterns
- **4625** reveals brute force attempts, credential stuffing, misconfigured service accounts

Filtering tip: Combine with `LogonType` to find anomalous access patterns (e.g., batch job at 3 AM from foreign IP)

Log Source: Nginx

Format: Combined Apache Log

```
192.168.1.50 - - [05/May/2024:14:30:00 +0000] "GET /checkout HTTP/1.1" 200 1234 "-" "Mozilla/5.0"
```

Enriched Fields:

- `clientip` → GeolP lookup (country, location)
- `timestamp` → Parsed via date filter
- `status` → HTTP status code
- `request` → URL path and method

Log Source: SSH

Format: JSON (structured)

```
{  
  "timestamp": "2024-05-05T14:30:00Z",  
  "event": "ssh_auth",  
  "user": "admin",  
  "source_ip": "185.234.72.45",  
  "result": "failure"  
}
```

Key Insight: 80% successful logins (internal IPs) vs 20% failures (external attack IPs)

Log Source: Syslog (Linux)

Format: RFC5424

```
<34>1 2024-05-05T14:30:00Z webserver ssh 1234 - - Failed password for invalid user admin from 185.234.72.45 port 54321 ssh2
```

Parsed Fields:

- `priority` → severity + facility
- `hostname` → origin system
- `process` → service name
- `message` → the actual event

Log Source: Syslog (Cisco)

Format: CEF (Common Event Format)

```
CEF:0|Cisco|IOS|12.4|5|SSH login attempt|3|src=192.168.1.100 dst=10.0.0.5 spt=54321 dpt=22
```

Fields: Device type, severity level, source/destination IPs, ports

Log Source: App Service

Format: JSON (structured)

```
{  
  "timestamp": "2024-05-05T14:30:00Z",  
  "method": "POST",  
  "endpoint": "/api/checkout",  
  "status_code": 200,  
  "latency_ms": 145  
}
```

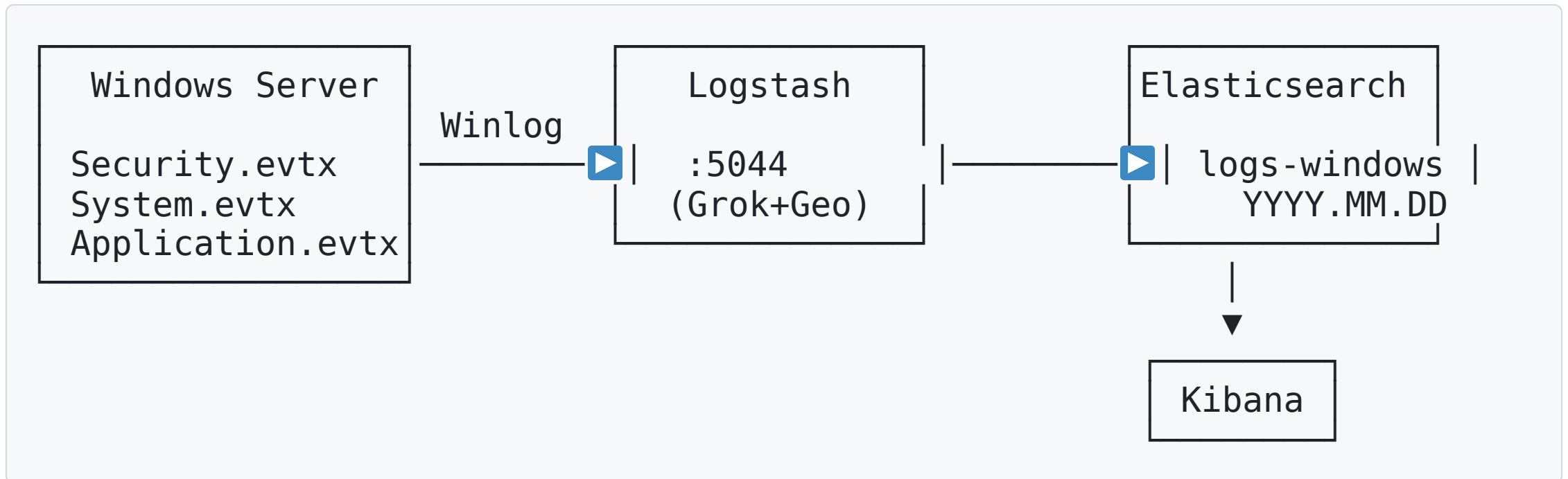
Metrics Tracked: Response times, error rates, endpoint popularity

Setting Up: Windows Endpoints

Winlogbeat: Official Elastic Shipper for Windows Events

Winlogbeat monitors Windows Event Logs and streams them to Logstash.

Architecture:



Full winlogbeat.yml Configuration:

Setting Up: Start the Stack

Step 1: Navigate to docker directory

```
cd elk-lab/docker
```

Step 2: Launch all services

```
docker compose up -d
```

Step 2: Wait for initialization (30-60 seconds)

```
docker compose ps
```

Step 3: Verify services are running

```
curl http://localhost:9200          # Elasticsearch  
curl http://localhost:5601/api/status # Kibana
```

Accessing Kibana

Open in browser:

```
http://localhost:5601
```

First Time Setup:

1. Click "Explore on my own" (skip tutorial)
2. Stack Management → Index Patterns
3. Create patterns for each log source

Configuring Kibana: Index Patterns

Create these patterns in order:

Pattern	Time Field
logs-nginx-*	@timestamp
logs-ssh-*	@timestamp
logs-syslog-*	@timestamp
logs-cisco-*	@timestamp
logs-app-*	@timestamp
logs-windows-*	@timestamp

Note: You can also use logs-* to query all sources at once

Configuring Kibana: Create Visualizations

Recommended Visualizations:

1. **SSH Attack Map** (Maps)

- Layer: Terms on `geoip.location`
- Source: `logs-ssh-*`

2. **SSH Auth Results** (Pie Chart)

- Slice by: Terms → `ssh_result`

3. **Nginx Traffic Over Time** (Line Chart)

- X-axis: Date histogram → `@timestamp`

4. **HTTP Status Codes** (Bar Chart)

- X-axis: Terms → `status`

Build a Dashboard

Steps:

1. Click **Dashboard** → **Create dashboard**
2. Add all saved visualizations
3. Arrange and resize panels
4. Save as "ELK Lab Overview"

Pro Tip: Use `logs-*` pattern to see all data, or filter by specific source using the `service` field

Generating Test Data

Nginx Logs (web traffic)

```
curl http://localhost:80  
curl http://localhost:80/products/123  
curl http://localhost:80/checkout
```

SSH Logs (security events)

```
docker exec ssh-simulator /generate_ssh_logs.sh 50  
# Generates 80% success (internal) + 20% failure (external)
```

Generating Test Data (continued)

Syslog Events

```
docker exec syslog-simulator /generate_syslog.sh 100  
# 50 Linux RFC5424 + 50 Cisco CEF events
```

App Logs (API requests)

```
curl http://localhost:5000/api/health  
curl -X POST http://localhost:5000/api/checkout \  
  -H "Content-Type: application/json" \  
  -d '{"item":"test","user_id":"user-123}"'
```

Verifying Data Arrived

In Kibana Discover:

1. Click **Discover** (left sidebar)
2. Select `logs-*` index pattern
3. Set time range to "Last 15 minutes"
4. You should see documents appearing

Via Command Line:

```
curl localhost:9200/_cat/indices?v
```

Look for indices like: `logs-nginx-2024.05.05`

Logstash Pipeline: How It Works

```
input { beats { port => 5044 } } # Accepts BOTH Filebeat AND Winlogbeat

filter {
  # NGINX: Parse Apache Combined Log format
  if [service] == "nginx" {
    grok { match => { "message" => "%{COMBINEDAPACHELOG}" } }
    geoip { source => "clientip" target => "geoip" }
    mutate { add_field => { "[@metadata][index]" => "nginx" } }
  }

  # SSH: Parse JSON with country codes
  if [service] == "ssh" {
    json { source => "message" target => "ssh_data" }
    date { match => [ "[ssh_data][timestamp]" "ISO8601" ] }
    geoip { source => "[ssh_data][source_ip]" target => "geoip" }
    mutate { rename => { "[ssh_data][result]" => "ssh_result" }
      add_field => { "[@metadata][index]" => "ssh" } }
  }

  # WINDOWS: Route Security events by Event ID
  if [winlog][channel] == "Security" {
    if [winlog][event_id] == 4624 {
      mutate { add_field => { "logon_result" => "success"
        "[@metadata][index]" => "windows" } }
    }
    if [winlog][event_id] == 4625 {
      mutate { add_field => { "logon_result" => "failure"
        "[@metadata][index]" => "windows" } }
    }
  }

  # GeoIP on source IP for attack mapping
  geoip { source => "[winlog][network][ip_address]" target => "geoip" }
}

# SYSLOG LINUX: RFC5424 format
if [service] == "syslog" {
  grok { match => { "message" => "<%(POSINT:priority)>..." } }
  mutate { add_field => { "[@metadata][index]" => "syslog" } }
}

# CISCO: CEF format
if [service] == "cisco" {
  grok { match => { "message" => "CEF:%{NOTSPACE}..." } }
  mutate { add_field => { "[@metadata][index]" => "cisco" } }
}

# APP: JSON API logs
if [service] == "app" {
  json { source => "message" target => "app_data" }
  mutate { rename => { "[app_data][latency_ms]" => "request_latency" }
    add_field => { "[@metadata][index]" => "app" } }
}
}

output {
  elasticsearch { index => "logs-%{[@metadata][index]}-%{+YYYY.MM.dd}" }
}
```

ELK Stack Components

Shippers: Filebeat & Winlogbeat

- **Filebeat:** Ships logs from Linux/Windows files
- **Winlogbeat:** Ships Windows Event Logs (Security, System, Application)

Logstash Pipeline

- **Input:** Receives from Beats on port 5044
- **Filter:** Grok parsing + GeolP enrichment
- **Output:** Routes to Elasticsearch indices by service

Elasticsearch

- **Inverted Index:** Enables sub-millisecond search across terabytes
- **Daily indices:** `logs-{service}-YYYY.MM.DD`

Error Budgets & SLOs

99.95% SLO → 21.6 min/month allowed downtime

$$43,200 \text{ min} \times (1 - 0.9995) = 21.6 \text{ min/month}$$

Why p95/p99? Mean hides outliers. p99 = 99% of users get acceptable latency.

The "SRE Clamp": When budget >50% consumed → freeze feature deployments.

SLI Proposal

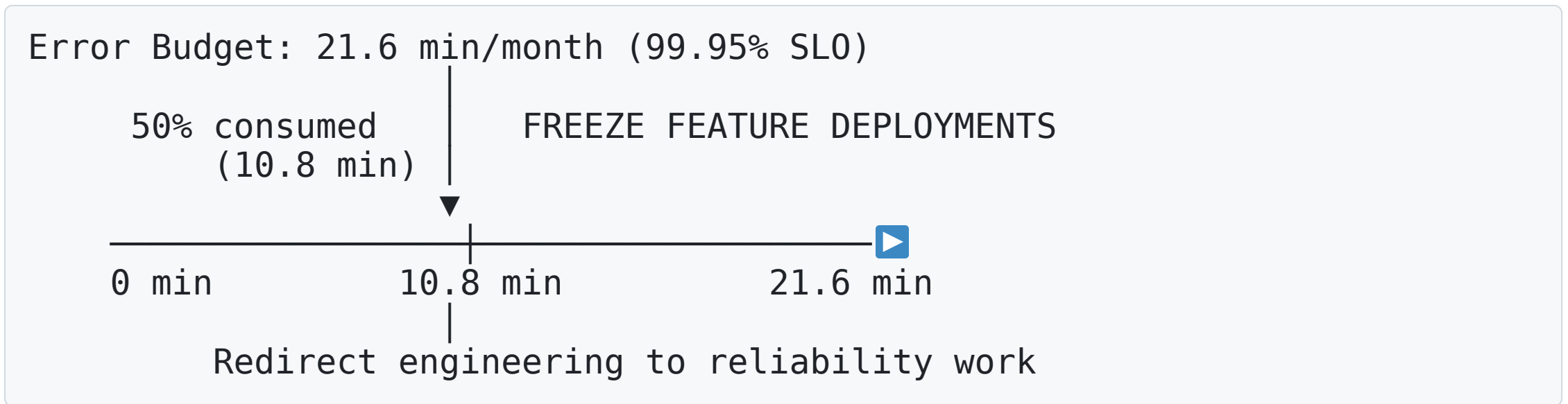
Proposed SLI: Percentage of `POST /api/checkout` returning HTTP 2xx AND latency < 500ms

Why this SLI:

- 5xx = complete failure (user can't complete purchase)
- Cart abandonment rises sharply above 400ms
- Checkout is the money moment (critical user journey)

Measurement: `count(http_status >= 200 && http_status < 300 && latency < 500) / count(*)`

The SRE Clamp



Rule: When >50% of error budget consumed (>10.8 of 21.6 min), freeze feature deployments until stability improves.

Key Commands Reference

```
# Start/stop
docker compose up -d
docker compose down

# View logs
docker compose logs -f filebeat
docker compose logs -f logstash

# Test Elasticsearch
curl localhost:9200/_cat/indices?v

# Generate data
docker exec ssh-simulator /generate_ssh_logs.sh 50
docker exec syslog-simulator /generate_syslog.sh 100

# Restart a service
docker compose restart filebeat
```

Windows Event IDs (Security Log)

Troubleshooting

No data in Kibana?

```
# Check Filebeat
docker exec filebeat filebeat test config
docker logs filebeat

# Check Logstash
docker compose logs logstash | tail -50

# Verify indices exist
curl localhost:9200/_cat/indices?v
```

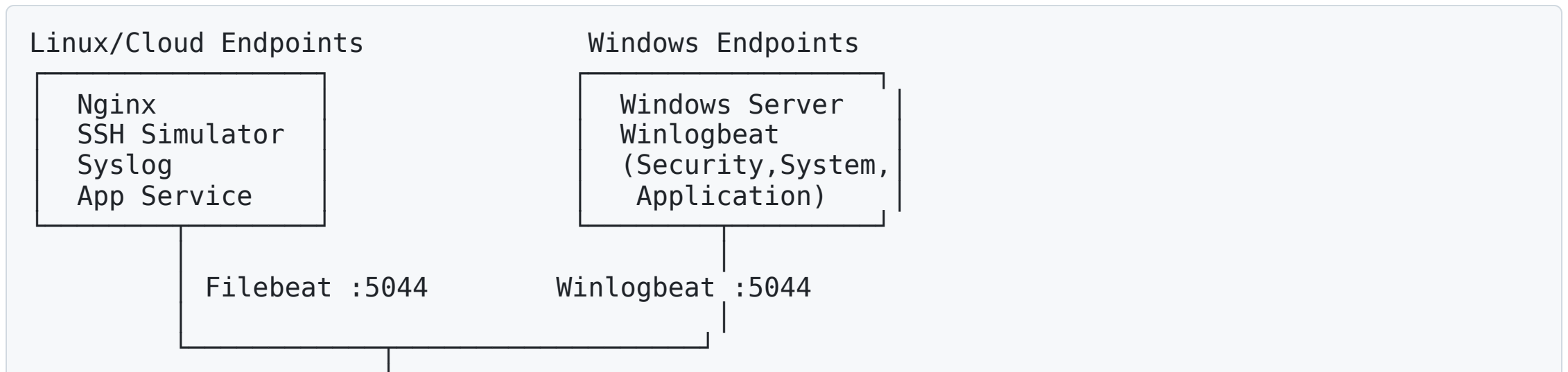
Services not healthy?

```
docker compose restart
docker compose logs --tail=100
```

System Interaction Maps

Data Source Pipelines:

Source	Pipeline
Nginx	Filebeat → Logstash (Grok) → ES → Kibana
SSH	Filebeat → Logstash (JSON) → ES → Kibana
Windows	Winlogbeat → Logstash → ES → Kibana



Summary

Component	Purpose	Key Config
Filebeat	Ship logs from files	filebeat.yml
Logstash	Parse & enrich	logstash.conf
Elasticsearch	Store & index	docker-compose.yml
Kibana	Visualize	Index patterns

Next Steps: Explore the data, create custom visualizations, build your own dashboard!

Questions?

Lab Repository: <https://github.com/H3xKatana/elk-lab>

Backup: File Structure

```
elk-lab/
├── docker/
│   ├── docker-compose.yml
│   ├── nginx.conf
│   ├── generate_ssh_logs.sh
│   ├── generate_syslog.sh
│   └── app/
├── config/
│   ├── filebeat.yml
│   └── logstash.conf
├── logs/
│   ├── nginx/
│   ├── ssh/
│   ├── syslog/
│   └── app/
├── scripts/
│   └── generate_answer_sheet.py
├── docs/
│   ├── presentation.md
│   ├── student_answer_sheet-2.md
│   └── README.md
└── html/
```